

A practical, end-to-end recipe for solving an ML problem

1) Frame the problem & success metric

- **What are we predicting?** Classification, regression, ranking, etc.
- **Define “done”** with a measurable metric and constraints (latency, memory, fairness).

Example: Predict if a loan will default (binary classification).

Metric: F1 (class imbalance).

Fun example: Predict pizza delivery time (regression) so the app shows an ETA.

Formulas (common):

- $\text{Accuracy} = \frac{TP+TN}{TP+FP+TN+FN}$
- $\text{Precision} = \frac{TP}{TP+FP}, \text{Recall} = \frac{TP}{TP+FN}$
- $F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
- $\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$

Intuition: Your metric steers every decision; pick it to match the real cost of errors.

Applications: Fraud detection (recall), recommender systems (NDCG), forecasts (MAE/RMSE).

Diffs: Unlike pure research, applied ML starts with a **business/impact metric** first.

2) Understand & collect data (with ethics)

- Identify features, labels, leakage risks, sampling bias, privacy/fairness needs.

Example: Gather past loans with repayment outcome, applicant features, macro features.

Intuition: Garbage in → garbage out; represent the deployment population.

Diffs: Deep learning often needs *more* data; classical ML can work well with less.

3) Split data properly

- **Train/Validation/Test** (e.g., 70/15/15). Use **time-based splits** for forecasting; **grouped splits** to avoid leakage across users/items.

Intuition: Validation tunes; test estimates *future* performance.

Diff: In small data, prefer **k-fold CV** (see Step 7).

4) Build a simple baseline

- Majority class, linear/logistic regression, or a simple heuristic.

Example: Always predict “on-time” pizza → gets you a floor to beat.

Intuition: Baselines expose whether complexity is actually helping.

Diff: In deep learning tasks, a **pretrained backbone** is a modern “strong baseline.”

5) Clean & explore (EDA)

- Handle missing values, outliers, inconsistent units; visualize distributions; check target leakage.

Intuition: Most real gains come from understanding quirks of the data.

6) Feature engineering (for classic ML) / Data preprocessing (for DL)

- **Classic ML:** create ratios, counts, interactions, encodings.
- **DL:** normalize, tokenize, augment, resize; labels/annotation quality.

Example: For loans: debt-to-income ratio, recent-inquiries count.

Pizza fun: Distance, rush-hour flag, rain indicator.

Diff: DL shifts complexity from “features” to **architecture + data scale/aug.**

7) Choose models & validation scheme

- Try a small set: linear/logistic, tree ensembles (RF/XGBoost), and if appropriate, neural nets.
- **Validation:** k-fold CV for tabular/small data; stratified for class imbalance.

Formula (k-fold CV risk):

$$(\hat{R} = \frac{1}{K} \sum_{k=1}^K \frac{1}{|V_k|} \sum_{i \in V_k} \ell(y_i, \hat{y}_i^{(-k)}))$$

Intuition: Reliable validation reduces overfitting and surprises at deployment.

Diffs: For sequences/vision/NLP, SOTA often = **pretrained transformers** fine-tuned.

8) Train, regularize, and tune

- **Losses (common):**
 - Regression: MSE ($= \frac{1}{n} \sum (y - \hat{y})^2$)
 - Classification: Cross-entropy ($= -\frac{1}{n} \sum [y \log \hat{p} + (1-y) \log(1-\hat{p})]$)
- **Regularization:** L2 ($+\lambda |w|_2^2$), L1 ($+\lambda |w|_1$), dropout, early stopping.
- **Tuning:** grid/random/Bayesian search; tune a few high-impact hyperparams first.

Intuition: Bias–variance trade-off—regularization reduces variance at some bias cost.

9) Evaluate on the test set (once!)

- Report metric + confidence intervals; inspect calibration and error slices (by user group, geography, time).

Example (calibration idea): Reliability curves to compare predicted vs empirical probabilities.

Diffs: In imbalanced problems, prefer PR curves over ROC.

10) Error analysis & iteration

- Manually audit top errors; label issues → data fixes; feature gaps → new signals; model issues → try alt models.

Intuition: Systematically turning errors into hypotheses is the fastest improvement loop.

11) Deploy & monitor (MLOps)

- Package (API/batch), add **drift detection**, **logging**, **feedback loop** for re-training, **A/B tests**, rollback plan.

Common drifts: covariate shift ($P(X)$), prior shift ($P(Y)$), concept shift ($P(Y|X)$).

Diffs: Research stops at test; production adds **monitoring**, **alerting**, **retraining cadence**.

12) Document & communicate

- Track dataset versions, code, metrics, decisions, ethical review, and risks.